

DIGITALSPY

Predictive Cyber Defence

Implementation Specification — Exact Week-1 Build Contract

NTRO / Smart India Hackathon 2026

Internal team use • Not for publication

1. Purpose and contract

This document is the coding source of truth for the one-week DigitalSpy prototype. It intentionally excludes broad research discussion and keeps only the decisions required to implement, test, integrate, and demo the system. Do not change core parameters because a test result looks unattractive; changes are allowed only through a documented experiment.

Definition of Done: CSV/PCAP → 24-D state S_t → Logistic Regression + Markov baseline → Attention-LSTM → five-horizon risk/tactic forecast → ATT&CK context → SHAP/attention evidence → local Ollama agent → Streamlit → chronological benchmark.

2. Scope: ships this week

Component	Status
CIC-IDS2017 primary benchmark	IMPLEMENT
Flow-level aggregation	IMPLEMENT
Packet-derived features from small PCAP sample	IMPLEMENT
10-second windows, per source host	IMPLEMENT
24-D S_t	IMPLEMENT
Six-state Z_t	IMPLEMENT
Exact label precedence	IMPLEMENT
Logistic Regression	IMPLEMENT
First-order Markov model	IMPLEMENT
Attention-LSTM	IMPLEMENT
K=5 direct forecasts	IMPLEMENT
Risk head + tactic head	IMPLEMENT
SHAP + temporal attention evidence	IMPLEMENT
ATT&CK mapping	IMPLEMENT
Offline Ollama agent	IMPLEMENT
Streamlit UI	IMPLEMENT
Next-target/link prediction	PHASE 2
TGN/GNN	PHASE 2
Mamba/SSM	PHASE 2
Full Transformer	PHASE 2

3. Locked mathematical specification

$S_t \in \mathbb{R}^{24}$
 $Z_t \in \{\text{BENIGN, RECON, INITIAL_ACCESS, LATERAL_MOVEMENT, C2, IMPACT}\}$
 $H_t = [S_{t-19}, \dots, S_t]$
 $K = 5$
step = 10 seconds
forecast horizon = 50 seconds

4. Feature schema

#	Feature	Type / aggregation
1	flow_count	count
2	total_bytes	sum

3	total_packets	sum
4	mean_flow_duration	mean
5	bytes_per_sec	derived
6	packets_per_sec	derived
7	syn_ratio	ratio
8	ack_ratio	ratio
9	fin_ratio	ratio
10	rst_ratio	ratio
11	psh_ratio	ratio
12	mean_iat	mean
13	std_iat	std
14	max_iat	max
15	mean_packet_len	mean
16	std_packet_len	std
17	distinct_dst_ips	distinct count
18	distinct_dst_ports	distinct count
19	distinct_src_ports_used	distinct count
20	fwd_bwd_byte_ratio	ratio
21	mean_init_win_bytes_fwd	mean
22	ttl_variance	packet-derived
23	tcp_window_std	packet-derived
24	fragmentation_ratio	packet-derived

5. Raw-label precedence implementation

```
RANK = {
    "IMPACT": 1, "C2": 2, "LATERAL_MOVEMENT": 3,
    "INITIAL_ACCESS": 4, "RECON": 5, "BENIGN": 6
}
```

```
BUCKETS = {
    DoS/GoldenEye/slowloris/Slowhttptest/DDoS: IMPACT,
    Bot: C2,
    Infiltration: LATERAL_MOVEMENT,
    Web Attack / FTP-Patator / SSH-Patator / Heartbleed: INITIAL_ACCESS,
    PortScan: RECON,
    BENIGN: BENIGN
}
```

tactic(window) = bucket with minimum rank among labels present

Normalize label whitespace and perform matching against the actual strings found in the local data. Save a versioned mapping file and unit-test every raw label category.

6. File verification before any model training

```
import pandas as pd, glob
for f in sorted(glob.glob('cic2017/*.csv')):
    df = pd.read_csv(f, usecols=lambda c: c.strip() == 'Label')
    print(f, df.iloc[:,0].str.strip().value_counts().to_dict())
```

Create and save: `verified_files.json`, `label_counts.json`, SHA256 hashes, and the final split manifest. The actual files are the source of truth; do not hard-code a remembered schedule without verification.

7. Train / validation / test

Split	Data	Use
Train	Monday–Wednesday local files	Fit models + scalers + transition matrix
Validation	Thursday local files	Early stopping, threshold, hyperparameters
Test	Friday local files	Final evaluation only

Hard rule: No random row-level split for the main result. Do not fit preprocessing on the entire dataset.

8. Window construction algorithm

1. Sort records by timestamp within each source host.
2. Create fixed 10-second non-overlapping bins.
3. Aggregate flow rows into each source-host/window.
4. Drop windows with fewer than two flows.
5. Attach packet-derived PCAP features where a matching source-host/window exists.
6. Apply training-only normalization parameters.
7. Store timestamp, source host, 24-D S_t , raw labels, mapped Z_t , and target metadata.

9. Sequence construction

```
For each source host:
  ordered states =  $S_1 \dots S_T$ 
  for t in 20 ... T-K:
     $X = [S_{\{t-19\}}, \dots, S_t]$ 
    targets = {risk_{t+1..t+5}, tactic_{t+1..t+5}}
```

Prevent sequences from crossing an invalid temporal gap. If the source host has no usable window for a required timestep, either break the sequence or explicitly implement a mask; do not silently stitch distant observations together.

10. Tier 1 — Logistic Regression

```
Input:  $S_t$ 
Output:  $P(\text{attack at } t)$ 
Metrics: precision, recall, macro-F1, FPR
```

Use class weighting. Tune no more than necessary on validation data. Save coefficients, scaler, seed, and evaluation predictions.

11. Tier 2 — Markov model

Input: Z_t

Estimate P_{ij} from training transitions

Forecast: $p_{t+k} = p_t P^k$

Use the same six-state taxonomy and the same chronological training period. Save `transition_matrix.csv` and a reproducible script that generates it.

12. Tier 3 — Attention-LSTM

Setting	Exact value
Input dimension	24
History	20 timesteps
LSTM layers	1
Hidden size	64
Dropout	0.20
Attention	additive / Bahdanau-style
Forecast horizons	5
Optimizer	Adam
Learning rate	$1e-3$
Batch size	64
Loss	weighted BCE + weighted CE, summed across horizons
Early stopping	Thursday validation loss

12.1 Forward computation

$h_1 \dots h_{20} = \text{LSTM}(S_{t-19} \dots S_t)$

$e_i = v^T \tanh(W_h h_i + b_h)$

$\alpha_i = \text{softmax}(e_i)$

$c = \sum \alpha_i h_i$

$\text{risk}_k = \text{sigmoid}(\text{head_risk}(c))$

$\text{tactic}_k = \text{softmax}(\text{head_tactic}(c))$ for $k=1..5$

13. Loss and class imbalance

$L_{\text{risk}} = \sum_k \text{weighted_BCE}(y_{\text{risk},k}, \hat{y}_{\text{risk},k})$

$L_{\text{tactic}} = \sum_k \text{weighted_CE}(y_{\text{tactic},k}, \hat{y}_{\text{tactic},k})$

$L_{\text{total}} = L_{\text{risk}} + L_{\text{tactic}}$

Do not oversample individual windows for the primary experiment. Use loss weighting so sequence neighbourhoods remain intact. Report class support counts and macro-F1.

14. Forecast evaluation

$F1_k = \text{macro-F1 for horizon } k$

$F1_K = (F1_1 + F1_2 + F1_3 + F1_4 + F1_5) / 5$

Metric	Definition
Macro-F1	Average class F1 across six tactic classes

FPR	False positives / all actual negatives for defined binary task
Hit@K	Phase-2 only for next-target prediction
Lead time	attack onset – first valid forecast crossing
Brier	Mean squared probabilistic error
ECE	Calibration gap by confidence bins

15. Threshold and lead-time procedure

1. Use Thursday validation data to select the risk threshold.
2. Freeze the threshold.
3. On Friday test data, scan forecasts chronologically.
4. For each attack instance, find the first valid forecast crossing before onset.
5. Compute lead time = onset – forecast timestamp.
6. Report median lead time and percentage of test instances with positive lead time.

16. Success criteria

1. Attention-LSTM macro-F1 $\geq$ Logistic Regression macro-F1 + 5 percentage points, while FPR does not rise by more than 2 percentage points.
2. Attention-LSTM horizon-averaged tactic macro-F1 $\geq$ Markov horizon-averaged macro-F1 + 5 percentage points.
3. Median positive forecast lead time $\geq$ 10 seconds on $\geq 60\%$ of test attack instances.

17. Explainability implementation

SHAP is used to produce feature-level evidence. The LSTM attention module exposes α_i temporal weights. The UI should show top features and high-attention windows without claiming causation.

18. ATT&CK mapping implementation

forecasted project bucket $\rightarrow$ deterministic ATT&CK context lookup $\rightarrow$ tactic / technique description

Keep the mapping in a versioned local JSON/STIX-compatible resource. Avoid internet calls during the demo.

19. Local agent implementation

Item	Choice
Runtime	Ollama
Model target	3B–4B Q4; 7B–8B optional
Controller	Plain Python
Output source	Structured forecast JSON
Tools	recent flows, host history, ATT&CK lookup, explain forecast, what-if, report
Internet	Not required

Autonomous blocking	No
---------------------	----

forecast.json → agent → tool calls → summary.md / UI card

The agent must read, not rewrite, the numerical forecast. Validate tool-call arguments and use a schema for the structured forecast.

20. Streamlit UI

Panel	Content
Input	Upload CSV or select demo sample
Current state	Source host, timestamp, key features
Risk	Current / future risk curve
Forecast	t+1...t+5 risk + tactic
Evidence	Top SHAP features + attention windows
ATT&CK	Predicted tactic / likely technique context
Agent	Analyst summary and investigation buttons
What-if	Baseline vs scenario risk curves
Benchmark	Model comparison table

21. Suggested repository structure

```

digitalspy/
  data/
    raw/
    processed/
    manifests/
  src/
    ingestion/
    features/
    labels/
    states/
    baselines/
    markov/
    lstm/
    explainability/
    attack_mapping/
    agent/
    simulator/
    ui/
    evaluation/
  configs/
  models/
  reports/
  tests/
  notebooks/
  scripts/
  README.md

```

22. Unit tests that must exist

- Label precedence: every raw label maps to the expected bucket.
- Multi-label window: highest-priority rank wins.
- BENIGN: only when no attack label appears.
- Windowing: timestamps exactly on bin boundaries are assigned consistently.
- Sequence builder: each X has exactly 20 timesteps and 24 features.
- Target builder: horizons are exactly +1...+5.
- Scaler: fit only on training data.
- Markov matrix: each nonempty row sums to 1.
- Forecast JSON: contains risk[1..5] and tactic[1..5].
- Agent: cannot overwrite forecast values.

23. Daily implementation plan

Day	Build	Gate
1	Dataset audit + file verification + manual flow/scan inspection	Verified manifests and label counts
2	Feature pipeline + packet-derived fusion + windows	Reloadable state dataset
3	Logistic + Markov	Baseline metrics + 6×6 matrix
4	Attention-LSTM	Stable validation training + saved checkpoint
5	K-step evaluator + ATT&CK + XAI	Forecast JSON + evidence
6	Streamlit + Ollama agent	Offline end-to-end run
7	Final benchmark + reproducibility + demo rehearsal	Final results package

24. Phase-2 architecture

Current: Feature-vector state → Attention-LSTM

Phase 2: Temporal graph $G_t \rightarrow$ TGN/TGNN → sequence dynamics (GRU/Mamba) → next-target link prediction

25. Demo runbook

1. Load a held-out traffic scenario.
2. Show 20-window history.
3. Display $t+1...t+5$ risk and tactic forecasts.
4. Show SHAP and attention evidence.
5. Open ATT&CK context.
6. Ask the local agent for a summary.
7. Run one bounded what-if scenario.
8. Finish with the benchmark table and lead-time result.

26. What not to do

- Do not use random train/test rows for the headline result.
- Do not tune on Friday.
- Do not fabricate metrics.
- Do not call attention weights causal.
- Do not let the LLM generate numerical probabilities.
- Do not add TGN/Mamba/Transformer to Week 1 unless the core system is already complete and evaluated.
- Do not add next-target prediction just to make the diagram look more advanced.

27. Reproducibility checklist

- Dataset source, local path, file hashes.
- Verified file list and label counts.
- 24-feature schema version.
- Label mapping version.
- Windowing parameters.
- Train/validation/test manifests.
- Training-only normalization statistics.
- Random seeds.
- Model hyperparameters.
- Saved weights.
- Raw predictions.
- Validation threshold.
- Evaluation scripts and outputs.
- Environment/package lock file.

28. Final engineering decision

Freeze the architecture here. Further discussion should concern implementation quality, data validity, evaluation, or Phase-2 research—not adding another model family to the Week-1 core. The project becomes stronger when every layer can be explained line by line by the team.